Object Oriented Programming (CSEG1044)
Unit 04 – Lecture 02 Notes
Generic Methods + Bounds + Wildcards

April 1, 2026

Contents

1 Lecture Snapshot **2**

2 Lecture Overview **2**

3 Prerequisite Bridge **2**

4 Learning Outcomes **2**

5 Suggested Reading Order **3**

6 Key Terms (Quick) **3**

7 Detailed Notes **3**
- 7.1 Generic methods . . . 3
- 7.2 Bounds (restrict allowed types) . . . 4
- 7.3 Wildcards: making collection parameters flexible . . . 4
  - 7.3.1 `? extends T` (producer/read) . . . 4
  - 7.3.2 `? super T` (consumer/write) . . . 4

8 Running Project Connection **4**

9 Common Mistakes and Confusions **5**

10 Worked Example: generic methods + bounds + wildcards **5**

11 Demo / Observation Task **6**

12 Practice Check (with brief solutions) **6**

13 Self-Study Practice (no solutions) **6**

14 Further Study **6**

15 Exit Questions (with sample answers) **7**

1 Lecture Snapshot

- **Unit focus:** Generics, Lambdas, Swing, and JDBC
- **Main new ideas:** Generic methods, Bounds (restrict allowed types), Wildcards: making collection parameters flexible
- **Course thread:** Use Generic Methods + Bounds + Wildcards to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

2 Lecture Overview

In the previous lecture you learned generic classes like `Box<T>`. Now we learn three ideas that appear frequently in real Java code:

- **generic methods**: `<T>` methods that work for many types,
- **bounds**: restricting which types are allowed (`extends`),
- **wildcards**: flexible method parameters for collections (`? extends`, `? super`).

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

3 Prerequisite Bridge

Before reading the new material in depth, do a fast recall of the older ideas listed below.

- **Generic classes**: This lecture extends the first generics lecture from type declarations to more flexible operations. Main reference: `Unit 04 / Lecture 01 slides/notes`.
- **Inheritance**: Bounds and wildcards only make sense if subtype relationships are already familiar. Main reference: `Unit 02 / Lecture 04 slides/notes`.

This lecture only revisits those ideas briefly so that the main explanation can stay focused on the new concept sequence.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

4 Learning Outcomes

After this lecture, you should be able to:

- Write a generic method and explain why it reduces duplication.
- Use bounds (`extends`) to restrict allowed types.
- Use wildcards (`? extends`, `? super`) with collections.

5 Suggested Reading Order

1. Read the **Prerequisite Bridge** first and confirm each recalled idea.
2. Study the central explanation in this order: *Generic methods → Bounds (restrict allowed types) → Wildcards: making collection parameters flexible.*
3. Trace the worked example line by line before checking short answers.
4. Attempt the practice section without looking at the solution immediately.
5. Finish with the exit questions and further-study suggestions to confirm retention.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

6 Key Terms (Quick)

- **Generic method**: method with its own type parameter, e.g., `static <T> T first(List<T> list)`.
- **Bound**: restriction on type parameter, e.g., `<T extends Number>`.
- **Wildcard**: unknown type parameter, e.g., `List<?>`.
- **Upper-bounded wildcard**: `? extends Type` (read/produce).
- **Lower-bounded wildcard**: `? super Type` (write/consume).
- **Invariance**: `List<Integer>` is not a subtype of `List<Number>`.

7 Detailed Notes

7.1 Generic methods

Sometimes only one method needs to be generic (not the whole class). Example: a method that prints any array.

Syntax:

- type parameter list comes before return type: `public static <T> void print(T[] a)`.

7.2 Bounds (restrict allowed types)

Without bounds, `T` can be any reference type. With bounds, you restrict it:

- `<T extends Number>` means `T` must be `Number` or a subclass (`Integer`, `Double`, ...).
- Common pattern: `<T extends Comparable<T>»` to support comparisons.

7.3 Wildcards: making collection parameters flexible

The key beginner confusion is:

- `List<Integer>` is **not** a `List<Number>` (invariance).

So how do we write a method that can accept `List<Integer>` *or* `List<Double>`? We use wildcards.

7.3.1 `? extends T` (producer/read)

If a method only needs to **read** from a list, use `? extends`. Example: sum of numbers:

- `List<? extends Number>` accepts `List<Integer>`, `List<Double>`, ...

Limitation: you cannot safely add new elements (except `null`) because the exact subtype is unknown.

7.3.2 `? super T` (consumer/write)

If a method needs to **add** elements of type `T` to a list, use `? super T`. Example: add integers to a list that can store integers or any of their supertypes.

PECS (memory trick). Producer Extends, Consumer Super.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

8 Running Project Connection

Use Generic Methods + Bounds + Wildcards to move Campus Resource Manager toward a real desktop application with UI, callbacks, and persistence.

- Use Campus Resource Manager as the running case study, or map the same idea to your own project domain if you are using another example.
- Ask where today's idea should live: model, service, DAO, UI, or team workflow.
- Use the lecture example as a smaller version of the same design choice you will make in the capstone.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC

Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

9 Common Mistakes and Confusions

- Using a wildcard when a named type parameter would be clearer.
- Overengineering beginner APIs with too many bounds.
- Assuming List<Child> is automatically a List<Parent>.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

10 Worked Example: generic methods + bounds + wildcards

Generic method (print array)

```
public class GenericDemo {
    public static <T> void printArray(T[] a) {
        for (T x : a) System.out.print(x + " ");
        System.out.println();
    }
}
```

Bounded type parameter (max)

```
public static <T extends Comparable<T>> T max(T a, T b) {
    return (a.compareTo(b) >= 0) ? a : b;
}
```

Upper-bounded wildcard (sum)

```
import java.util.List;

public static double sum(List<? extends Number> list) {
    double s = 0.0;
    for (Number n : list) s += n.doubleValue();
    return s;
}
```

Lower-bounded wildcard (add)

```
import java.util.List;

public static void addOnes(List<? super Integer> list, int count) {
    for (int i = 0; i < count; i++) list.add(1);
}
```

11 Demo / Observation Task

Use this block as a short reproducible demo or observation task.

- Reproduce the lecture's main example around *Generic methods* once without looking at the solution first.
- Change one input, rule, or design choice in Campus Resource Manager and predict the result before checking it.
- Record one success case, one edge case, and one failure case so the idea becomes observable instead of purely theoretical.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

12 Practice Check (with brief solutions)

1. Why is `List<Integer>` not a subtype of `List<Number>`?

   Answer: generics are invariant in Java; allowing it would break type safety (you could add a `Double` into a `List<Integer>` through a `List<Number>` reference).

2. If you only need to read from a list, which wildcard is common: `extends` or `super`?

   Answer: `? extends ...`.

3. What is the PECS rule?

   Answer: Producer Extends, Consumer Super.

13 Self-Study Practice (no solutions)

1. Write a generic method `swap(T[] a, int i, int j)`.
2. Write `min` using `<T extends Comparable<T>`.
3. Write a method that copies from one list to another using wildcards (hint: one produces, one consumes).

14 Further Study

- Revisit the prerequisite references if any recall bullet still feels weak.
- Rework the lecture example with one changed assumption, input, or design choice.
- Read the textbook and documentation references listed in the source line for a second explanation of the same topic.

- Apply the same idea once inside Campus Resource Manager so that the concept moves from theory to design practice.

Sources: Schildt, *Java: The Complete Reference* (12e) (generics, lambdas, Swing, JDBC).; Bloch, *Effective Java* (3e) (generics + lambdas).; Oracle Java Tutorials: Generics + Lambda Expressions + Swing + JDBC Basics (accessed April 1, 2026).; Java SE API docs (e.g., `java.util.function`, `javax.swing`, `java.sql`) (accessed April 1, 2026).

15 Exit Questions (with sample answers)

Q1. When do you use `? extends T`?

Sample answer: Use `? extends T` when your method only needs to read values of type `T` from a structure and doesn't need to add elements, because it can accept multiple subtype lists safely.

Q2. What does a bound like `<T extends Number>` mean?

Sample answer: It means the type parameter `T` must be `Number` or a subclass of `Number`, so you can safely use `Number`-level operations inside the method/class.

Q3. What problem do bounds solve in generic APIs?

Sample answer: They restrict allowed types so the compiler can guarantee valid operations inside the API.